# Technical Design Document (TDD) - ForgeQuote

## Enterprise RFQ Costing, Estimation Governance & Intelligence Platform

### 1. Document Information

| Attribute | Specification |
|---|---|
| Project Name | ForgeQuote (Automotive & Industrial RFQ Costing Workspace) |
| Document Title | Enterprise Technical Design Document (TDD) - Core Build & Production Architecture |
| Document Version | 2.1.0-PROD-SPEC |
| Author | Principal Software Architect & Senior Staff Systems Engineer (Company A Pairing Lead) |
| Creation Date | November 2026 |
| Current Date / Review Date | 2026-11-09 |
| Document Status | Final Engineering Review & Production Transition Baseline |
| Intended Audience | Lead Architects, Principal Engineers, VP of Engineering, Estimating Operations Leads, Commercial Sales Leadership, Head of Plant Operations, Compliance & IT Security Officers |
| Repository Baseline | https://github.com/MoAftaab/isocodelabs |

#### Revision History

| Version | Date | Author / Role | Summary of Changes |
|---|---|---|---|
| 0.1.0 | 2026-11-02 | Lead Estimating Architect | Initial core build prototype specification (single part, SQLite, initial cost formulas). |
| 1.0.0 | 2026-11-04 | Senior Staff Engineer | Multi-part RFQ support, Alembic schema migrations, and basic AI assistant integration. |
| 1.5.0 | 2026-11-06 | Systems Architect | Multi-operation machine routing, effective-dated rate masters, and sales return workflow. |
| 2.0.0 | 2026-11-08 | Principal Architect | Private Supabase Storage bucket integration, standalone GST snapshotting, and 1,400-row workbook import. |
| 2.1.0 | 2026-11-09 | Senior Staff Engineer | Comprehensive Enterprise Technical Design Document covering Low-Level Design (LLD), deep data flows, statutory governance, threat analysis, and production cloud architecture. |

### Live Deployment and Verified Reviewer Access

| Component | Provider | Live URL | Verified status |
|---|---|---|---|
| Frontend application | Vercel | https://frontend-seven-tau-20.vercel.app | HTTP 200; application page served. |
| Backend and API | Render | https://forgequote-api-aftaab.onrender.com | HTTP 200; FastAPI service responds. |
| API health | Render | https://forgequote-api-aftaab.onrender.com/api/health | HTTP 200; PostgreSQL and Supabase Storage connected. AI reports unavailable. |
| Interactive API documentation | Swagger UI | https://forgequote-api-aftaab.onrender.com/docs | HTTP 200; interactive OpenAPI docs load. |

Both reviewer accounts were verified through the Render backend and the Vercel frontend proxy. Sign-in, authenticated **/api/me**, and sign-out succeeded for both roles.

| Role | Username | Password |
|---|---|---|
| Estimator | **estimator@demo.local** | **estimate-demo-2026** |
| Sales Head | **sales@demo.local** | **approve-demo-2026** |

These are intentional reviewer demo credentials for the synthetic sample workspace. Do not reuse them for production access. The health response currently reports AI unavailable; no successful live AI inference is claimed by this deployment status.

## 2. Executive Summary

### 2.1 What is the System?

ForgeQuote is an enterprise-grade Request for Quotation (RFQ) estimation, costing, and commercial governance platform engineered specifically for tier-1 closed-die forging and precision machining manufacturers (exemplified by Company A, Chakan, Pune). The system ingests multi-part RFQ packages, digitizes technical specifications from engineering drawings and commercial workbooks, models multi-operation manufacturing routings, executes deterministic price calculations across multiple volume scenarios, enforces statutory approval thresholds, and dispatches legally binding cost-breakup quotation documents.

### 2.2 What Problem Does It Solve?

Industrial component estimation in mid-to-large precision forging enterprises frequently suffers from fragmented, error-prone spreadsheets, manual calculation drift, unversioned labor/machine hourly rates, and informal quotation approvals. These systemic weaknesses lead to:

1. Significant profit margin erosion caused by uncaptured raw material yield scrap or outdated machine center rates.

2. Unauditable quote histories and lack of defensible records during customer open-book cost breakdowns.

3. Severe operational bottlenecks where senior commercial leadership is forced to review low-margin or non-viable bids without contextual historical intelligence.

4. Non-compliance with corporate delegation-of-authority matrices regarding large capital commitments (>₹25 Lakhs).

ForgeQuote replaces ad-hoc spreadsheets with a unified, stateful web workspace that enforces mathematical determinism, temporal rate consistency, strict separation of duties between Engineering Estimators and the Sales Head, and complete provenance tracking from raw drawing ingestion to post-quotation win/loss analysis.

### 2.3 Primary Capabilities

- Multi-Modal RFQ Ingestion: Ingests vector PDFs, scanned drawing images (PNG/JPEG/TIFF/BMP), plain text/email files, and multi-sheet Excel workbooks (**.xlsx**), backed by hybrid persistence (local disk or private Supabase Storage buckets).
- Dual-Mode Intake Intelligence: Employs local heuristics and regex-based pattern extraction for offline, zero-trust environments, while providing an optional OpenAI/Anthropic-compatible AI gateway for structured extraction constrained strictly by source-text substring verification.
- Deterministic Multi-Operation Cost Engine: Powered by Python **decimal.Decimal** with explicit half-up currency quantization, computing gross material yield loss, ordered machine center routing steps, fixed overhead/freight/packing adders, markup

margins, and standalone statutory Goods and Services Tax (GST).

- Multi-Scenario Volume Modeling: Calculates and compares cost breakdowns simultaneously across two or three distinct annual volume points (e.g., 10,000, 50,000, and 180,000 units/year).
- Temporal Rate Management: Effective-dated master pricing tables for raw materials (₹/kg) and machine centers (₹/hour) that automatically maintain non-overlapping validity intervals.
- Immutable Revision Snapshots & Four-Eyes Governance: Automatically creates frozen quote revisions capturing formula versions, rate snapshots, and GST percentages. Enforces corporate role separation where estimators draft and submit revisions, while the Sales Head reviews, approves, or returns them with mandatory recorded feedback.
- Statutory ₹25L Governance & Audit-Proof PDF Dispatch: Flags quotes exceeding the ₹25 Lakh statutory annual value threshold, generates customer-facing cost-breakup quotation PDFs using ReportLab, and tracks downstream dispatch (**sent**) and commercial outcomes (**won**/**lost** with mandatory win/loss reasons).
- Historical Quote Intelligence & Mapped Bulk Ingestion: Provides sub-second paginated search across historical and active quotes, attribute-based routing suggestion heuristics, and a 2-step mapped Excel workbook importer capable of previewing and committing over 1,400 historical rows with complete batch provenance.

### 2.4 What Makes Its Architecture Technically Interesting?

1. Mathematical Determinism vs. Assistive AI Separation: ForgeQuote establishes a non-negotiable architectural boundary: AI models are strictly confined to passive text extraction, conversational Q&A, and historical routing ranking. All financial calculations, margins, taxations, and approvals are executed deterministically by Python server code.

2. Contractual NDA Privacy Guardrails: The platform enforces an explicit data boundary through **AI_NDA_APPROVED**. If enterprise zero-data-retention (ZDR) terms are unverified, customer RFQ data is strictly blocked from external cloud gateways; inference is confined to synthetic demo records, and intake automatically falls back to local regex extraction and local Tesseract OCR.

3. Dual Database & Hybrid Storage Strategy: Seamlessly operates across zero-dependency local SQLite and enterprise cloud PostgreSQL (Supabase) via SQLAlchemy 2.0 and Alembic, accompanied by a storage abstraction that pivots between local disk and authenticated Supabase Object Storage buckets.

4. Point-in-Time Temporal Rate Snapshots: By storing denormalized JSON snapshots of the exact material rates, machine rates, operation parameters, and tax percentages within each quote revision, the platform guarantees that historical quotes remain mathematically immutable even when global rate masters are escalated years later.

### 2.5 Major Engineering Challenges

- Scanned Drawing Extraction Quality: OCR on noisy technical blueprints containing dimension callouts, geometric dimensioning and tolerancing (GD&T) symbols, and dense title blocks without specialized CAD vision pipelines.
- In-Memory Session Scalability: Transitioning from the current single-process in-memory session and import staging architecture to a distributed cache (Redis) capable of supporting multi-node horizontal scaling.
- Workbook Format Heterogeneity: Accommodating varied customer and legacy historical Excel formats through dynamic column mapping and streaming validation without blowing the server heap.

## 3. Problem Statement & Motivation

### 3.1 Existing Problem & Business Requirement

Company A operates a high-capacity closed-die forging and precision machining facility in the Chakan industrial corridor of Pune, India. The company manufactures mission-critical powertrain, drivetrain, and chassis components-such as steering knuckles, hydraulic lift arms, gear blanks, axle pins, and PTO coupling hubs-for major domestic and international automotive OEMs.

The commercial quoting lifecycle at Company A involves responding to high-velocity Requests for Quotation (RFQs). Each RFQ typically bundles 1 to 50 distinct engineered parts, each requiring specific raw material alloys (e.g., EN8, EN19, EN24, 20MnCr5, 40Cr4 forging steel), gross billet weights ranging from 0.3 kg to 25 kg, varying flash scrap allowances, multi-step production routings (shearing $\rightarrow$ induction heating $\rightarrow$ hammer/press forging $\rightarrow$ trimming $\rightarrow$ normalizing $\rightarrow$ CNC turning $\rightarrow$ precision grinding), and multi-tiered annual production volumes (e.g., prototype 5,000; ramp-up 25,000; full-scale 180,000 units/year).

### 3.2 Limitations of Existing Approaches

Historically, Company A relied on localized Microsoft Excel templates maintained independently by individual estimators:

- Calculation Drift & Formula Corruption: Complex Excel workbooks with embedded macros frequently suffered from accidental formula overwrites, circular references, and inconsistent rounding methods (e.g., truncating vs. half-up rounding).
- Temporal Rate Inconsistency: Rate masters (steel prices, energy tariffs, machine hour costs) were manually hardcoded or referenced from stale network drives. When steel prices surged from ₹118/kg to ₹142/kg, quotes in progress failed to reflect updated costs, leading to commercial underquoting.
- Lack of Governance & Rogue Quoting: Quotes were dispatched directly to clients via email as unversioned PDFs or raw spreadsheets without formal sign-off from the Sales Head. High-value commitments (>₹25 Lakhs) bypassed senior commercial review.
- Data Leakage & Compliance Exposure: Passing customer engineering blueprints and proprietary CAD specifications into unvetted public cloud AI services poses catastrophic risks of intellectual property theft and breach of customer NDAs.
- Dark Historical Data: Over 1,400 legacy quotes remained trapped in static, unindexed workbooks, preventing estimators from leveraging prior routing cycles or benchmarking win/loss pricing.

### 3.3 Proposed Solution

ForgeQuote provides a centralized, web-native estimation cockpit that unifies RFQ intake, document parsing, deterministic multi-operation costing, temporal rate resolution, four-eyes approval workflows, and historical intelligence into an audit-proof system.

```
+-----------------------------------------------------------------------------------------------+
| FORGEQUOTE WORKSPACE LIFECYCLE                                                                |
|                                                                                               |
| [ Customer RFQ ]                                                                              |
|                 |                                                                             |
|                 ▼                                                                             |
| [ Multi-Modal Intake ] ──► (Local Tesseract OCR / Rule Extract / Guarded AI)                  |
|                 |                                                                             |
|                 ▼                                                                             |
| [ Part & Routing Spec ] ──► (Ordered Operations: Turning, Forging, Grinding @ Sequence 1..N)  |
|                 |                                                                             |
|                 ▼                                                                             |
| [ Deterministic Engine ] ──► (Python Decimal: Yield Scrap + Machine Hours + Adders + Margin %)|
|                 |                                                                             |
|                 ▼                                                                             |
| [ Frozen Revision ] ──► (Rate Snapshots + JSON Breakdown + GST Rate Snapshot)                 |
```

| FORGEQUOTE WORKSPACE LIFECYCLE |
|---|
| \| |
| ▼ |
| [ Sales Governance ] ──► (Approve / Return with Mandatory Feedback + Statutory ₹25L Rule) |
| \| |
| ▼ |
| [ Controlled PDF ] ──► (Audit-Proof Breakdown + GST Separation + Dispatch Tracking) |

+----------+

```

### 3.4 Expected Technical & Business Outcomes

- 100% Elimination of Rounding and Calculation Errors: Guaranteed arithmetic determinism via Python **Decimal** across all quotes.
- 80% Reduction in Quoting Turnaround Time: Automated document parsing and historical routing suggestions compress intake and estimation cycles from days to hours.
- Zero NDA Breaches: Contractual enforcement of data privacy boundaries, ensuring proprietary customer data never touches hosted LLMs without explicit authorization.
- Complete Audit Traceability: Every quote revision permanently freezes its constituent rates, formulas, taxes, and approvals, providing immutable evidence for open-book OEM audits.
- Statutory Compliance: Enforced four-eyes review and mandatory flagging for all quotes exceeding ₹25,00,000 annual volume.

## 4. Goals and Non-Goals

```

+----------+

| SYSTEM BOUNDARIES MATRIX |
|---|

+----------+----------+

| GOALS | NON-GOALS |
|---|---|
| • Deterministic Decimal-precision costing | • Full-scale Enterprise Resource Planning (ERP) |
| • Multi-operation routing & rate versioning | • Direct CNC G-code or machine tool telemetry |
| • Strict NDA privacy guards for hosted AI | • Unchecked autonomous AI quoting or approval |
| • Four-eyes sales governance & PDF generation | • Direct SMTP/Outlook email dispatch |
| • Dual SQLite / Supabase PostgreSQL support | • 3D CAD parametric geometry reconstruction |

| • Deterministic Decimal-precision costing | • Full-scale Enterprise Resource Planning (ERP) |
|---|---|
| • 1,400+ row historical quote bulk ingestion | • Supplier portal for raw ingot procurement |
| • Statutory ₹25L approval rule enforcement | • Complex multi-currency hedging / Forex engine |

+----------+----------+

```

### 4.1 Functional Goals

- Provide an intuitive, responsive single-page application (SPA) tailored for manufacturing estimators and the Sales Head.
- Support multi-part RFQ bundles containing up to 100 line items per RFQ package.
- Allow estimators to define up to 12 ordered machining/forging operations per part with discrete machine center rate keys.
- Calculate unit costs, markups, unit quote prices, annual values, and separate GST across 2 or 3 distinct volume scenarios simultaneously.
- Freeze calculations into immutable **QuoteRevision** records with comprehensive JSON breakdowns and rate snapshots.
- Provide a dedicated Approvals workspace for the Sales Head to approve quotes or return them with mandatory written feedback.
- Restrict generation of official ReportLab cost-breakup PDFs to approved revisions only.
- Ingest and parse vector PDFs, scanned drawing images, plain text emails, and multi-worksheet Excel workbooks.
- Enable conversational search and historical part similarity suggestions using a grounded AI assistant or deterministic attribute matching.

### 4.2 Technical Goals

- Maintain zero calculation divergence between test environments and production through pure Python **Decimal** arithmetic.
- Support zero-configuration local development via SQLite, while offering full cloud production readiness via Supabase PostgreSQL and private object storage.
- Guarantee sub-100ms response times for core API queries (RFQ listing, costing, rate retrieval) under typical concurrency.
- Ensure strict database consistency via ACID transactions and Alembic database migration scripts.
- Implement defensive input validation on all API endpoints using Pydantic schemas.

### 4.3 Performance Goals

- Single-part quote calculation execution: $\le 15\text{ ms}$.
- RFQ register list retrieval (100 RFQs with joins): $\le 80\text{ ms}$.
- Historical search query response (1,500+ records): $\le 120\text{ ms}$.
- ReportLab quotation PDF generation: $\le 350\text{ ms}$.
- Batch historical workbook import (1,400 rows): $\le 3.5\text{ seconds}$.

### 4.4 Security Goals

- Isolate sessions using cryptographically secure tokens stored in **HttpOnly**, **SameSite=Strict** cookies.
- Defend against Cross-Site Request Forgery (CSRF) on all mutating endpoints using constant-time double-submit token validation.
- Store passwords exclusively as PBKDF2-HMAC-SHA256 digests salted with 16 cryptographically random bytes across 310,000 iterations.
- Prevent confidential customer data leakage to public AI endpoints via hardcoded system-level policy checks.
- Prevent unauthenticated access to uploaded RFQ technical drawings and specifications.
```

### 4.5 Features Explicitly Outside Scope (Non-Goals)

- Direct ERP / MRP Synchronization: ForgeQuote does not automatically push bills of materials (BOMs) into SAP, Oracle, or Microsoft Dynamics in this Core Build.
- Direct Mail Server Dispatch: The system does not send emails via SMTP; the Sales Head manually dispatches the downloaded quotation PDF and marks the quote **sent** in the platform.
- Direct Machine Telemetry / SCADA: The system does not connect to shop-floor IoT sensors or PLC controllers to measure real-time spindle cycle times.
- Automated CAD Feature Recognition: The system does not parse native STEP/IGES/DWG 3D models to compute part volumes automatically; estimators review 2D drawings and input weight and cycle times.
- Managing Director / Plant Head Sign-Off Chains: The current build implements the core Estimator $\leftrightarrow$ Sales Head workflow; higher-tier executive approvals are deferred to roadmap releases.

## 5. Requirements

### 5.1 Functional Requirements Matrix

| ID | Priority | Category | Requirement Description | Source Verification |
|---|---|---|---|---|
| FR-01 | P0 | Costing | Calculate unit material cost adjusted for gross yield scrap: $C_{\text{mat}} = \frac{W_{\text{gross}} \times R_{\text{mat}}}{1 - S}$. | [**backend/costing.py**](file:///backend/costing.py#L23) |
| FR-02 | P0 | Costing | Calculate processing cost across multiple ordered routing operations: $C_{\text{proc}} = \sum \frac{T_i}{60} \times R_{\text{machine}, i}$. | [**backend/costing.py**](file:///backend/costing.py#L24) |
| FR-03 | P0 | Costing | Calculate unit cost ($C_{\text{mat}} + C_{\text{proc}} + C_{\text{fixed}}$) and quote price using percentage markup on cost: $P = C \times (1 + M)$. | [**backend/costing.py**](file:///backend/costing.py#L26-L31) |
| FR-04 | P0 | Costing | Enforce boundary validation: $W_{\text{gross}} > 0$, $T \ge 0$, $0 \le S < 100$, $0 \le M < 100$, positive non-zero whole integer volumes. | [**backend/costing.py**](file:///backend/costing.py#L19-L30) |
| FR-05 | P0 | Costing | Multi-scenario evaluation across 2 or 3 distinct volume points with automatic reference volume determination (max volume). | [**backend/main.py**](file:///backend/main.py#L471-L504) |
| FR-06 | P0 | Governance | Freeze quote calculations into immutable **QuoteRevision** records with rate snapshots and JSON breakdowns. | [**backend/main.py**](file:///backend/main.py#L505-L514) |
| FR-07 | P0 | Governance | Role separation: Estimators submit drafts (**draft** $\rightarrow$ **pending_approval**); Sales Head approves or returns with mandatory reason. | [**backend/main.py**](file:///backend/main.py#L522-L573) |
| FR-08 | P0 | Governance | Statutory threshold flag: detect and audit quotes exceeding ₹25,00,000 annual value at reference volume before GST. | [**backend/main.py**](file:///backend/main.py#L562-L570) |

| ID | Priority | Category | Requirement Description | Source Verification |
|---|---|---|---|---|
| FR-09 | P0 | Output | Generate professional, audit-proof ReportLab quotation PDFs showing cost breakups, separate GST, and synthetic disclaimers. | [**backend/main.py**](file:///backend/main.py#L575-L618) |
| FR-10 | P0 | Rate Master | Maintain temporal rate versions for materials and machines with non-overlapping effective dates and automatic period closing. | [**backend/main.py**](file:///backend/main.py#L441-L465) |
| FR-11 | P1 | Ingestion | Multi-format upload parser supporting text, email, CSV, openpyxl Excel, vector PDF, and scanned OCR drawings. | [**backend/main.py**](file:///backend/main.py#L902-L959) |
| FR-12 | P1 | Ingestion | Local Tesseract OCR processing for scanned PDFs and raster drawings (up to 20 pages max). | [**backend/main.py**](file:///backend/main.py#L961-L973) |
| FR-13 | P1 | Intake AI | Extract structured RFQ fields with strict requirement that every field cite an exact substring from the source document. | [**backend/main.py**](file:///backend/main.py#L1008-L1035) |
| FR-14 | P1 | Privacy | Enforce **AI_NDA_APPROVED** boundary: block customer RFQ data from hosted inference; restrict AI to synthetic demo records. | [**backend/main.py**](file:///backend/main.py#L701-L743) |
| FR-15 | P1 | History | Sub-second paginated search across active parts and imported historical quotes with text filtering. | [**backend/main.py**](file:///backend/main.py#L653-L690) |
| FR-16 | P1 | History | 2-step mapped Excel importer with worksheet selection, auto column matching, validation preview, and batch commit. | [**backend/main.py**](file:///backend/main.py#L1274-L1402) |
| FR-17 | P2 | Storage | Hybrid file storage: private Supabase Object Storage bucket when configured, with fallback to local **instance/uploads/**. | [**backend/main.py**](file:///backend/main.py#L815-L850) |
| FR-18 | P2 | Commercial | Sales Head lifecycle actions: mark quote **sent**, record final outcome (**won** or **lost** with mandatory loss reason). | [**backend/main.py**](file:///backend/main.py#L620-L651) |

### 5.2 Non-Functional Requirements (NFR) Matrix

| Category | Requirement Specification | Metric / Validation Standard | Architecture Enforcement |
|---|---|---|---|
| Scalability | Support 100+ concurrent estimator sessions; ingest workbooks with 10,000+ rows without memory exhaustion. | P99 latency $< 200\text{ ms}$ at 50 req/sec; streaming openpyxl iteration. | Stateless API design, paginated queries, database connection pooling. |
| Reliability | Zero financial calculation drift; 100% repeatable quote figures across server restarts and OS platforms. | Identical output to 2 decimal places using **ROUND_HALF_UP**. | Python **Decimal** engine in **costing.py**; automated boundary unit tests. |
| Availability | Core estimation, costing, and PDF dispatch must function 100% offline without external cloud dependencies. | Offline operational capability when AI gateway or internet is severed. | Local SQLite fallback, local regex extraction, local ReportLab PDF engine. |
| Maintainability | Clean separation of concerns; zero business logic in UI templates; declarative schema evolution. | Strict MVC/REST layering; 100% schema changes versioned via Alembic. | FastAPI routers, SQLAlchemy 2.0 ORM, modular CSS variables. |
| Security | Zero unauthorized access to confidential technical drawings; complete CSRF and session hijacking defense. | OWASP Top 10 compliance; constant-time token comparison; HttpOnly cookies. | **backend/security.py**, PBKDF2 (310k iterations), RBAC middleware. |
| Performance | Instantaneous UI feedback for numeric quote recalculation and scenario toggling. | Sub-15ms server execution time for **POST /api/rfqs/{id}/cost**. | Pre-compiled regexes, indexed foreign keys, optimized SQL joins (**joinedload**). |
| Observability | Complete immutable audit trail of all commercial operations, logins, rate modifications, and approvals. | Every state mutation recorded in **audit_events** with actor, action, and entity. | Synchronous audit logging hook in **main.py** invoked across all mutating routes. |
| Fault Tolerance | Malformed uploads or corrupted drawings must never crash the service or corrupt existing RFQ entities. | HTTP 413/415/422 responses; retention of original file bytes despite parse error. | Safe exception wrappers, size limits (12 MB), transaction rollbacks. |

## 6. High-Level Architecture

### 6.1 Overall Architectural Pattern

ForgeQuote is built on a Layered Service-Oriented Web Architecture with strict decoupled presentation, business logic, deterministic domain computation, and hybrid persistence tiers. The backend presents an asynchronous RESTful API built on FastAPI, serving both JSON endpoints and static SPA assets.

```mermaid

graph TB

subgraph ClientTier ["Presentation Tier (Browser Client)"]

SPA["Vanilla JS Responsive SPA<br/>(frontend/app.js)"]

UI_Theme["Manufacturing Design System<br/>(styles.css, feature-overrides.css)"]

SPA --- UI_Theme

end

subgraph GatewayTier ["Security & Ingress Tier (FastAPI Core)"]

SessionAuth["Session & Cookie Guard<br/>(HttpOnly, SameSite=Strict)"]

CSRFFilter["Double-Submit CSRF Filter<br/>(hmac.compare_digest)"]

RBACFilter["Role-Based Access Guard<br/>(Estimator vs. Sales Head)"]
```

```
AuditBus["Audit Logging Bus<br/>(audit_events)"]
end
subgraph ServiceTier ["Application & Domain Service Tier"]
CostEngine["Deterministic Cost Engine<br/>(backend/costing.py)"]
IntakeParser["Multi-Modal Document Parser<br/>(PyMuPDF, openpyxl, Tesseract)"]
AIGateway["AI Gateway Adapter<br/>(OpenAI / Anthropic Protocol)"]
PDFRenderer["Quotation Document Engine<br/>(ReportLab Platypus)"]
RateService["Temporal Rate Master<br/>(Effective Interval Resolver)"]
end
subgraph PersistenceTier ["Persistence & Cloud Storage Tier"]
subgraph RelationalDB ["Relational Store (SQLAlchemy 2.0)"]
SQLiteStore[("Local SQLite Engine<br/>instance/forgequote.db")]
PostgresStore[("Supabase Managed PostgreSQL<br/>(AWS ap-south-1 / Mumbai)")]
end
subgraph ObjectStorage ["Binary Document Store"]
LocalDisk["Local File Storage<br/>instance/uploads/"]
SupabaseStorage["Supabase Private Storage Bucket<br/>rfq-source-documents"]
end
end
subgraph ExternalEcosystem ["External Services Tier"]
LLMHost["Hosted Inference Gateway<br/>(AgentRouter / Lumosel / OpenAI)"]
end
SPA -->|HTTPS / REST API| SessionAuth
SessionAuth --> CSRFFilter --> RBACFilter --> AuditBus
AuditBus --> CostEngine
AuditBus --> IntakeParser
AuditBus --> AIGateway
AuditBus --> PDFRenderer
AuditBus --> RateService
CostEngine --> RateService
RateService --> RelationalDB
CostEngine --> RelationalDB
PDFRenderer --> RelationalDB
IntakeParser --> ObjectStorage
IntakeParser --> RelationalDB
AIGateway -->|Guarded HTTPS Call| LLMHost
AIGateway --> RelationalDB
```

### 6.2 Major System Components & Responsibilities

| Component Name | Architectural Module | Core Responsibilities |
|---|---|---|
| Presentation SPA | [**frontend/app.js**](file:///frontend/app.js), [**index.html**](file:///frontend/index.html) | Renders reactive views (RFQ Register, Costing Workspace, Rate Master, Approvals Queue, Quote History); handles optimistic UI updates, client-side input masking, modal management, and dynamic formatting (Indian Rupee **en-IN**, dates). |
| Security & Session Guard | [**backend/security.py**](file:///backend/security.py) | Manages 12-hour session lifecycles, computes PBKDF2 password digests, validates **X-CSRF-Token** headers, and enforces RBAC boundaries. |
| Deterministic Costing Service | [**backend/costing.py**](file:///backend/costing.py) | Pure domain module executing yield scrap modeling, multi-operation routing summations, fixed adders, markup applications, and half-up currency rounding. |
| Document Ingestion Pipeline | [**backend/main.py**](file:///backend/main.py#L869-L974) | Dispatches uploaded files to appropriate parsers (PyMuPDF for vector PDFs, openpyxl for workbooks, pytesseract for image drawings) and retains source files. |
| AI Gateway & Privacy Adapter | [**backend/main.py**](file:///backend/main.py#L1037-L1140) | Translates internal chat and extraction requests to OpenAI Chat Completions, OpenAI Responses, or Anthropic Messages protocols; enforces **AI_NDA_APPROVED** privacy firewall. |
| PDF Generation Engine | [**backend/main.py**](file:///backend/main.py#L575-L618) | Assembles approved quote breakdowns, rate snapshots, customer metadata, and tax figures into audit-proof PDF quotation documents via ReportLab. |
| Data Access & Migration Tier | [**backend/database.py**](file:///backend/database.py), [**backend/models.py**](file:///backend/models.py), [**migrations/**](file:///migrations/) | Manages database sessions, connection pooling, dialect auto-detection, relational object mapping, and versioned DDL migrations via Alembic. |

### 6.3 Communication Mechanisms & External Dependencies

- Client-to-Server: Asynchronous HTTP/1.1 and HTTP/2 JSON REST calls with **credentials: "same-origin"**. File uploads use **multipart/form-data**.
- Database Connectivity: SQLAlchemy 2.0 ORM over SQLite connection or PostgreSQL native wire protocol via **psycopg** (v3.2.9) with mandatory TLS (**sslmode=require**) on cloud connections.
- Object Storage Communication: HTTPS REST communication with Supabase Storage API (**/storage/v1/object/...**) authenticated via server-side service keys.
- AI Gateway Calls: Asynchronous HTTPS calls via **httpx.AsyncClient** with configurable connection timeouts (75 seconds) and fallback headers (**originator**, **User-Agent**).

## 7. Low-Level Design (LLD)

### 7.1 Module & Class Relationships

```mermaid
classDiagram
class Base {
<<DeclarativeBase>>
}
class User {
+int id
+str username
+str display_name
```

```
+str role
+str password_hash
+bool active
}
class RFQ {
+int id
+str rfq_number
+str customer
+date received_date
+date due_date
+int owner_id
+str status
+str loss_reason
+bool synthetic
+str source_label
+datetime created_at
+datetime updated_at
}
class Part {
+int id
+int rfq_id
+str part_name
+str material
+float gross_weight_kg
+int annual_volume
+float cycle_time_min
+str machine_rate_key
+float scrap_pct
+float overhead_per_part
+float freight_per_part
+float packing_per_part
+float margin_pct
+bool synthetic
+str source_ref
}
class RoutingOperation {
+int id
+int part_id
+int sequence_no
```

```
+str operation_name
+str machine_rate_key
+float cycle_time_min
}
class RateVersion {
+int id
+str rate_type
+str rate_key
+str unit
+float amount
+date effective_from
+date effective_to
+int editor_id
+bool synthetic
+datetime created_at
}
class QuoteRevision {
+int id
+int rfq_id
+int revision_number
+str status
+str scenarios_json
+str breakdown_json
+str rate_snapshot_json
+float annual_value
+float gst_rate_percent
+str formula_version
+int submitted_by
+datetime created_at
+datetime approved_at
+datetime sent_at
}
class Approval {
+int id
+int revision_id
+int reviewer_id
+str decision
+str reason
+datetime created_at
```

```
}
class RFQSource {
+int id
+str file_id
+int rfq_id
+int uploaded_by
+str original_filename
+str media_type
+int size_bytes
+str extraction_method
+str extraction_error
+datetime created_at
}
class HistoricalQuote {
+int id
+str batch_id
+int source_row
+str rfq_number
+str customer
+str part_name
+str material
+float gross_weight_kg
+int annual_volume
+float cycle_time_min
+str routing_key
+float quoted_unit_price
+float quoted_annual_value
+date quoted_at
+str source_label
+bool synthetic
+datetime created_at
}
class AuditEvent {
+int id
+int actor_id
+str action
+str entity_type
+str entity_id
+str detail
```

+datetime created_at

}

Base <|-- User

Base <|-- RFQ

Base <|-- Part

Base <|-- RoutingOperation

Base <|-- RateVersion

Base <|-- QuoteRevision

Base <|-- Approval

Base <|-- RFQSource

Base <|-- HistoricalQuote

Base <|-- AuditEvent

RFQ "1" *-- "many" Part : cascade delete-orphan

RFQ "1" *-- "many" QuoteRevision : cascade delete-orphan

RFQ "1" *-- "many" RFQSource : cascade delete-orphan

Part "1" *-- "many" RoutingOperation : cascade delete-orphan

QuoteRevision "1" *-- "many" Approval : tracks review history

User "1" <-- "many" RFQ : owner

User "1" <-- "many" QuoteRevision : submitted_by

User "1" <-- "many" Approval : reviewer

User "1" <-- "many" RFQSource : uploaded_by

User "1" <-- "many" RateVersion : editor

```

### 7.2 Function-Level Responsibilities & Algorithms

##### Costing Algorithm (**backend/costing.py**)

```python

def calculate_part(part: dict, material_rate: dict, machine_rate: dict,

volumes: list[int], route_rates: list[dict] | None = None) -> list[dict]:

## 1. Cast all physical and monetary quantities to high-precision Decimal

weight = Decimal(str(part["gross_weight_kg"]))

scrap = Decimal(str(part.get("scrap_pct", 0))) / Decimal("100")

margin = Decimal(str(part.get("margin_pct", 0))) / Decimal("100")

cycle = Decimal(str(part["cycle_time_min"]))

## 2. Resolve routing operations or fallback to base machine rate

routes = route_rates or [{"cycle_time_min": cycle, "amount": machine_rate["amount"]}]

## 3. Defensive boundary assertions

if weight <= 0 or cycle < 0 or not routes or not 0 <= scrap < 1 or not 0 <= margin < 1:

raise ValueError("Check weight, cycle time, scrap, and margin values.")

if Decimal(str(material_rate["amount"])) < 0 or any(Decimal(str(r["amount"])) < 0 for r in routes):

raise ValueError("Rates cannot be negative.")

## 4. Compute Gross Material Yield Cost (Yield loss equation)

material_cost = weight * Decimal(str(material_rate["amount"])) / (Decimal("1") - scrap)

## 5. Compute Summed Process Cost across all routing operations

process_cost = sum(Decimal(str(r["cycle_time_min"])) / Decimal("60") * Decimal(str(r["amount"]))

for r in routes)

## 6. Sum Fixed Adders (Overhead, Freight, Packing)

fixed_adders = sum(Decimal(str(part.get(k, 0)))

for k in ("overhead_per_part", "freight_per_part", "packing_per_part"))

## 7. Unit Cost and Scenario Expansion

unit_cost = material_cost + process_cost + fixed_adders

unit_price = unit_cost * (Decimal("1") + margin)

## 8. Half-Up Quantization to 0.01 precision

results = []

for volume in volumes:

v = int(volume)

results.append({

"volume": v,

"material": float(money(material_cost)),

"processing": float(money(process_cost)),

"overhead": float(money(part.get("overhead_per_part", 0))),

"freight": float(money(part.get("freight_per_part", 0))),

"packing": float(money(part.get("packing_per_part", 0))),

"unit_cost": float(money(unit_cost)),

"unit_price": float(money(unit_price)),

"annual_value": float(money(unit_price * v)),

})

return results

```

##### Temporal Rate Resolution Algorithm (**backend/main.py**)

To prevent historical quote mutation while enabling forward price adjustments:

$$\text{Rate}(K, T) = \arg\max_{R \in \text{RateVersion}} \{ R.\text{effective\_from} \mid R.\text{key} = K \land R.\text{effective\_from} \le T \land (R.\text{effective\_to} \ge T \lor R.\text{effective\_to} \text{ IS NULL}) \}$$
```

When inserting a new rate version starting at $T_{\text{new}}$, any existing rate version where $R.\text{effective_from} < T_{\text{new}}$ and $R.\text{effective_to} \ge T_{\text{new}}$ has its end date automatically updated to:

$$R.\text{effective_to} = T_{\text{new}} - 1\text{ day}$$

### 7.3 Design Patterns Applied

- Strategy Pattern (Document Extraction): **read_upload()** inspects the file suffix and MIME type, selecting the appropriate parsing strategy (**fitz**, **openpyxl**, **pytesseract**, or plain text decoder) dynamically.
- Snapshot Pattern (Quote Revisioning): Instead of storing live relational foreign keys to volatile master rates, **QuoteRevision** serializes the exact resolved rates, formulas, breakdowns, and GST percentages into immutable JSON blobs.
- Factory & Dependency Injection Pattern: FastAPI's **Depends(get_db)** and **Depends(get_current_user)** inject database sessions and authenticated user principals into route controllers with automatic cleanup.
- Two-Step Command Pattern (Workbook Import): Decouples parsing from persistence: **POST /api/imports/preview** validates and stores intermediate mapped data in a tokenized staging file; **POST /api/imports/{batch_id}/commit** validates database deduplication and commits the batch atomically.

## 8. Technology Stack & Rationale

```

+----------------------------------------------------------------------------------------------+

**TECHNOLOGY STACK ARCHITECTURE**

+--------------------+--------------------+----------------------------------------------------+

| CATEGORY | TECHNOLOGY | VERSION / RATIONALE |
|---|---|---|
| **Frontend Web** | **Vanilla ES6+ / HTML5** | **Native browser runtime; zero build tooling dependencies** |
| Frontend Styles | CSS3 Custom Props | Scoped variables, dark/light tones, zero CSS framework |
| Backend Framework | FastAPI | v0.115.14; High-performance ASGI, native Pydantic typing |
| ASGI Web Server | Uvicorn (standard) | v0.34.3; uvloop & httptools C-accelerated networking |
| ORM & Migrations | SQLAlchemy / Alembic | v2.0.41 / v1.16.2; Modern mapped typing, strict schema |
| Database Driver | psycopg (binary) | v3.2.9; High-performance PostgreSQL 15+ async driver |
| Document Parsing | PyMuPDF (fitz) | v1.24+; C-level MuPDF speed for PDF text & pixmap render |
| OCR Engine | Tesseract via PIL | pytesseract v0.3.13 / Pillow 10+; Local offline OCR |
| Spreadsheet Engine | openpyxl | v3.1.5; Standard ECMA-376 XLSX streaming reader/writer |
| PDF Report Engine | ReportLab | v4.4.2; Precision vector layouts, repeatable typography |
| HTTP Client | HTTPX | v0.28.1; Async/await connection pooling for AI gateway |
| Test Framework | Pytest | v8.4.1 / pytest-asyncio; Automated regression suites |
```

```
+--------------------+--------------------+-------------------------------------------------------+
```
```
```

## Detailed Technology Analysis

| Technology | Category | Version | Purpose in ForgeQuote | Why Selected Over Alternatives | Trade-Offs & Mitigations |
|---|---|---|---|---|---|
| FastAPI | Backend | **0.115.14** | REST API gateway, dependency injection, and static asset server. | Automatic OpenAPI docs, high async throughput, seamless integration with Pydantic v2 schemas. Selected over Flask (sync, lack of validation) and Django (unnecessary ORM bloat). | Requires explicit async threadpool management for CPU-bound OCR operations. |
| Uvicorn | Server | **0.34.3** | Lightning-fast ASGI production server. | Built on **uvloop** and **httptools**. Significantly out-benchmarks pure Python WSGI servers (Gunicorn default). | Single worker default in local development; must be configured with worker processes in production containers. |
| SQLAlchemy | Database | **2.0.41** | Declarative object-relational mapping, query building, connection pooling. | Industry standard in Python enterprise systems. The 2.0 mapped-column syntax provides static type checking with Mypy/Pyright. | Dynamic queries can be verbose; mitigated using explicit **joinedload** options to prevent N+1 query bugs. |
| psycopg (v3) | Database | **3.2.9** | Low-level PostgreSQL client library. | Pure rewrite supporting pipeline mode, native binary parameters, and modern Python type hints. Outperforms legacy **psycopg2**. | Binary wheels require specific platform builds (handled automatically via **psycopg[binary]**). |
| Alembic | Migration | **1.16.2** | Versioned relational DDL schema evolution. | Fully native to SQLAlchemy models. Tracks exact schema hashes (**0001** through **0004**) across SQLite and PostgreSQL. | Autogenerate must be inspected manually for column renames or complex index changes. |
| PyMuPDF (fitz) | Document | **1.24.14** | PDF text extraction and rasterization. | High-speed C library (**MuPDF**) capable of rendering high-DPI pixmaps for OCR 10x faster than pure-Python **pypdf**. | Heavy binary wheel dependency; codebase includes fallback to **pypdf** if fitz import fails. |
| pytesseract | OCR | **0.3.13** | Host OCR wrapper for scanned blueprints. | Connects to Google's open-source Tesseract OCR engine. Zero external SaaS costs. | Requires host-installed Tesseract binary; accuracy on noisy engineering drawings requires manual estimator verification. |

| Technology | Category | Version | Purpose in ForgeQuote | Why Selected Over Alternatives | Trade-Offs & Mitigations |
|---|---|---|---|---|---|
| openpyxl | Data | **3.1.5** | Ingesting and writing Excel workbooks. | Standard library for modern **.xlsx** workbooks. Supports **read_only=True** to parse 1,400+ row sheets with low RAM footprint. | Slower than C-based **calamine** or **polars**, but has zero compilation prerequisites. |
| ReportLab | Document | **4.4.2** | Generation of approved quotation PDFs. | Programmable PDF layout engine (**Platypus**) with exact point-level typography, table cell styling, and header pagination. | Verbose imperative styling code; mitigated by creating reusable document templates. |
| HTTPX | Network | **0.28.1** | Asynchronous communication with AI gateways. | Native **asyncio** support, robust connection pooling, and standard HTTP/2 transport. Selected over **requests** (synchro nous/blocking). | Strict connection timeouts must be set to prevent threadpool exhaustion during upstream LLM latency spikes. |

## 9. Detailed Data Flow

### 9.1 End-to-End Quotation Lifecycle

```mermaid
sequenceDiagram
autonumber
actor Estimator as Estimator (Aarav / Nisha)
participant UI as Responsive SPA (Browser)
participant API as FastAPI Backend (main.py)
participant Storage as Binary Store (Local / Supabase)
participant Costing as Cost Engine (costing.py)
participant DB as Database (PostgreSQL / SQLite)
actor Sales as Sales Head (Mira)
Note over Estimator, DB: Phase 1: RFQ Package Intake & Parsing
Estimator->>UI: Uploads Customer Drawing / RFQ Workbook
UI->>API: POST /api/intake/file-extract (multipart/form-data)
API->>Storage: Persist file bytes (instance/uploads or Supabase Bucket)
API->>API: Parse document (openpyxl / PyMuPDF / Tesseract OCR)
API->>DB: INSERT RFQSource (file_id, size, media_type, extraction_method)
API-->>UI: Return extracted fields & suggested part line-items
Estimator->>UI: Reviews & edits fields, clicks "Create RFQ"
UI->>API: POST /api/rfqs {rfq_number, customer, due_date, parts, source_file_ids}
API->>DB: INSERT RFQ & Part records (linked to RFQSource)
```

API->>DB: INSERT AuditEvent (create_rfq)

API-->>UI: 201 Created (RFQ register refreshed)

Note over Estimator, DB: Phase 2: Multi-Operation Routing & Costing

Estimator->>UI: Configures Part Cost Inputs (Weight, Scrap %, Operations, Margin %)

UI->>API: PATCH /api/parts/{part_id} {routing_operations: [Op1, Op2..]}

API->>DB: UPDATE Part & REPLACE RoutingOperation rows

API-->>UI: 200 OK (Updated part details)

Estimator->>UI: Selects Scenarios [10000, 50000, 180000] & clicks "Calculate Quote"

UI->>API: POST /api/rfqs/{rfq_id}/cost {volumes: [...]}

API->>DB: Query Effective Material & Machine Rates on current date

DB-->>API: Active RateVersion rows

API->>Costing: calculate_part() with Decimal arithmetic

Costing-->>API: Exact breakdown scenarios & annual values

API->>DB: INSERT QuoteRevision (status='draft', breakdown_json, rate_snapshot_json, gst_rate_percent)

API->>DB: INSERT AuditEvent (calculate_quote)

API-->>UI: 200 OK (Display Cost Breakdown Table & Scenarios)

Estimator->>UI: Clicks "Submit for Approval"

UI->>API: POST /api/revisions/{revision_id}/submit

API->>DB: UPDATE QuoteRevision & RFQ status to 'pending_approval'

API->>DB: INSERT AuditEvent (submit_for_approval)

API-->>UI: 200 OK (Revision submitted)

Note over Sales, DB: Phase 3: Sales Governance & PDF Dispatch

Sales->>UI: Navigates to Approvals Queue

UI->>API: GET /api/approvals

API->>DB: Query revisions WHERE status='pending_approval'

DB-->>API: Pending quote list

API-->>UI: Display items with breakdown & frozen rate snapshots

alt Sales Head Returns Quote

Sales->>UI: Enters return reason & clicks "Return"

UI->>API: POST /api/revisions/{revision_id}/decision {decision: 'return', reason: '...'}

API->>DB: INSERT Approval (decision='return', reason)

API->>DB: UPDATE QuoteRevision (status='returned'), RFQ (status='estimating')

API->>DB: INSERT AuditEvent (quote_return)

API-->>UI: 200 OK (Quote returned to estimator)

else Sales Head Approves Quote

Sales->>UI: Clicks "Approve Quote"

UI->>API: POST /api/revisions/{revision_id}/decision {decision: 'approve'}

API->>DB: Check annual value > ₹25L threshold flag

API->>DB: INSERT Approval (decision='approve')

```
API->>DB: UPDATE QuoteRevision (status='approved', approved_at=now), RFQ (status='approved')
API->>DB: INSERT AuditEvent (quote_approve)
API-->>UI: 200 OK (Approval confirmed, threshold_flag returned)
Sales->>UI: Clicks "Download Approved Quote PDF"
UI->>API: GET /api/revisions/{revision_id}/pdf
API->>DB: Fetch QuoteRevision, RFQ, Parts
API->>API: Render ReportLab Platypus document
API->>DB: INSERT AuditEvent (download_quote_pdf)
API-->>UI: Stream application/pdf
Sales->>UI: Dispatches to customer and clicks "Mark Sent"
UI->>API: POST /api/revisions/{revision_id}/sent
API->>DB: UPDATE QuoteRevision & RFQ status to 'sent'
API-->>UI: 200 OK
Sales->>UI: Records customer outcome (Won / Lost)
UI->>API: POST /api/rfqs/{rfq_id}/outcome {decision: 'lost', reason: 'Price higher than competitor'}
API->>DB: UPDATE RFQ status='lost', loss_reason='...'
API->>DB: INSERT AuditEvent (rfq_lost)
API-->>UI: 200 OK
end
```

## 10. Database Design

### 10.1 Relational Schema Architecture

The persistence layer is governed by 10 relational tables designed in Third Normal Form (3NF), with strategic JSON denormalization inside **quote_revisions** to achieve immutability.

```mermaid
erDiagram
users ||--o{ rfqs : "owns"
users ||--o{ rfq_sources : "uploads"
users ||--o{ quote_revisions : "submits"
users ||--o{ approvals : "reviews"
users ||--o{ rate_versions : "edits"
users ||--o{ audit_events : "triggers"
rfqs ||--o{ parts : "contains"
rfqs ||--o{ quote_revisions : "generates"
rfqs ||--o{ rfq_sources : "references"
parts ||--o{ routing_operations : "defines"
quote_revisions ||--o{ approvals : "receives"
users {
int id PK
```

```
varchar username UK "String(120), unique, index"
varchar display_name "String(120)"
varchar role "String(30), index"
varchar password_hash "String(300)"
boolean active "default true"
}
rfqs {
int id PK
varchar rfq_number UK "String(80), unique, index"
varchar customer "String(180), index"
date received_date
date due_date "index"
int owner_id FK "users.id"
varchar status "String(30), index"
text loss_reason "nullable"
boolean synthetic "default false"
varchar source_label "String(80)"
datetime created_at
datetime updated_at
}
parts {
int id PK
int rfq_id FK "rfqs.id, index, CASCADE"
varchar part_name "String(180)"
varchar material "String(120)"
float gross_weight_kg
int annual_volume
float cycle_time_min
varchar machine_rate_key "String(120)"
float scrap_pct "default 0"
float overhead_per_part "default 0"
float freight_per_part "default 0"
float packing_per_part "default 0"
float margin_pct "default 15"
boolean synthetic "default false"
text source_ref "nullable"
}
routing_operations {
int id PK
```

```
int part_id FK "parts.id, index, CASCADE"
int sequence_no "Unique(part_id, sequence_no)"
varchar operation_name "String(120)"
varchar machine_rate_key "String(120)"
float cycle_time_min
}
rate_versions {
int id PK
varchar rate_type "String(30), index"
varchar rate_key "String(120), index"
varchar unit "String(30)"
float amount
date effective_from "index, Unique(type, key, from)"
date effective_to "nullable"
int editor_id FK "users.id, nullable"
boolean synthetic "default false"
datetime created_at
}
quote_revisions {
int id PK
int rfq_id FK "rfqs.id, index, CASCADE"
int revision_number "Unique(rfq_id, revision_number)"
varchar status "String(30), index"
text scenarios_json
text breakdown_json
text rate_snapshot_json
float annual_value "default 0"
float gst_rate_percent "default 0"
varchar formula_version "String(30), default cost-v1"
int submitted_by FK "users.id, nullable"
datetime created_at
datetime approved_at "nullable"
datetime sent_at "nullable"
}
approvals {
int id PK
int revision_id FK "quote_revisions.id, index"
int reviewer_id FK "users.id"
varchar decision "String(30)"
```

```
text reason "nullable"
datetime created_at
}
rfq_sources {
int id PK
varchar file_id UK "String(100), unique, index"
int rfq_id FK "rfqs.id, nullable, index, CASCADE"
int uploaded_by FK "users.id, index"
varchar original_filename "String(255)"
varchar media_type "String(120)"
int size_bytes
varchar extraction_method "String(40), nullable"
text extraction_error "nullable"
datetime created_at
}
historical_quotes {
int id PK
varchar batch_id "String(80), index, Unique(batch_id, row)"
int source_row
varchar rfq_number "String(100), index"
varchar customer "String(180), index"
varchar part_name "String(180), index"
varchar material "String(120), index"
float gross_weight_kg "nullable"
int annual_volume "nullable"
float cycle_time_min "nullable"
varchar routing_key "String(120), nullable"
float quoted_unit_price "nullable"
float quoted_annual_value "nullable"
date quoted_at "nullable"
varchar source_label "String(180)"
boolean synthetic "default false"
datetime created_at
}
audit_events {
int id PK
int actor_id FK "users.id, nullable"
varchar action "String(80), index"
varchar entity_type "String(50)"
```

varchar entity_id "String(80)"

text detail

datetime created_at

}

```

### 10.2 Table Specifications, Keys, & Constraints

1. **users** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **username** is **UNIQUE** and indexed (**ix_users_username**).
   - Fields: **display_name** (VARCHAR 120), **role** (VARCHAR 30: **'estimator' | 'sales'**), **password_hash** (VARCHAR 300), **active** (BOOLEAN).
2. **rfqs** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **rfq_number** is **UNIQUE** and indexed (**ix_rfqs_rfq_number**).
   - Foreign Keys: **owner_id** $\rightarrow$ **users.id**.
   - Indexes: **due_date**, **status**, **customer**.
   - Fields: **received_date** (Date), **loss_reason** (Text, nullable), **synthetic** (Boolean), **source_label** (VARCHAR 80).
3. **parts** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Foreign Keys: **rfq_id** $\rightarrow$ **rfqs.id** ON DELETE CASCADE (indexed).
   - Fields: **part_name**, **material**, **gross_weight_kg**, **annual_volume**, **cycle_time_min**, **machine_rate_key**, **scrap_pct**, **overhead_per_part**, **freight_per_part**, **packing_per_part**, **margin_pct**, **synthetic**, **source_ref**.
4. **routing_operations** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Foreign Keys: **part_id** $\rightarrow$ **parts.id** ON DELETE CASCADE (indexed).
   - Constraints: **UniqueConstraint("part_id", "sequence_no", name="uq_part_route_sequence")**.
   - Fields: **operation_name**, **machine_rate_key**, **cycle_time_min**.
5. **rate_versions** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **UniqueConstraint("rate_type", "rate_key", "effective_from", name="uq_rate_key_start")**.
   - Foreign Keys: **editor_id** $\rightarrow$ **users.id** (nullable).
   - Indexes: **rate_type**, **rate_key**, **effective_from**.
   - Fields: **unit** (**₹ / kg** or **₹ / hour**), **amount** (Float), **effective_to** (Date, nullable).
6. **quote_revisions** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **UniqueConstraint("rfq_id", "revision_number", name="uq_rfq_revision")**.
   - Foreign Keys: **rfq_id** $\rightarrow$ **rfqs.id** ON DELETE CASCADE, **submitted_by** $\rightarrow$ **users.id**.
   - Fields: **revision_number** (Integer), **status** (VARCHAR 30: **draft**, **pending_approval**, **approved**, **returned**, **sent**), **scenarios_json** (Text), **breakdown_json** (Text), **rate_snapshot_json** (Text), **annual_value** (Float), **gst_rate_percent** (Float), **formula_version** (VARCHAR 30).
7. **approvals** Table:
```

- Primary Key: **id** (Integer, Auto-increment).
- Foreign Keys: **revision_id** $\rightarrow$ **quote_revisions.id** (indexed), **reviewer_id** $\rightarrow$ **users.id**.
- Fields: **decision** (**approve** | **return**), **reason** (Text, nullable), **created_at** (DateTime TZ).

8. **rfq_sources** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **file_id** is **UNIQUE** and indexed (**ix_rfq_sources_file_id**).
   - Foreign Keys: **rfq_id** $\rightarrow$ **rfqs.id** ON DELETE CASCADE (nullable), **uploaded_by** $\rightarrow$ **users.id**.
   - Fields: **original_filename**, **media_type**, **size_bytes**, **extraction_method**, **extraction_error**.
9. **historical_quotes** Table:
   - Primary Key: **id** (Integer, Auto-increment).
   - Constraints: **UniqueConstraint("batch_id", "source_row", name="uq_history_source_row")**.
   - Indexes: **batch_id**, **rfq_number**, **customer**, **part_name**, **material**.
   - Fields: **gross_weight_kg**, **annual_volume**, **cycle_time_min**, **routing_key**, **quoted_unit_price**, **quoted_annual_value**, **quoted_at**, **source_label**, **synthetic**.
10. **audit_events** Table:
    - Primary Key: **id** (Integer, Auto-increment).
    - Foreign Keys: **actor_id** $\rightarrow$ **users.id** (nullable).
    - Indexes: **action**.
    - Fields: **entity_type** (VARCHAR 50), **entity_id** (VARCHAR 80), **detail** (Text max 600 chars), **created_at** (DateTime TZ).

### 10.3 Migration Strategy (Alembic Version Graph)

The database schema evolution is managed through linear, idempotent Alembic migration scripts:

- **0001_initial** (**0001_initial_schema.py**): Establishes base tables (**users**, **rfqs**, **parts**, **rate_versions**, **quote_revisions**, **approvals**, **audit_events**, **historical_quotes**).
- **0002_routing_operations** (**0002_routing_operations.py**): Adds **routing_operations** table with composite unique constraint **uq_part_route_sequence** and foreign key cascade.
- **0003_rfq_sources** (**0003_rfq_sources.py**): Adds **rfq_sources** table for binary source file tracking, extraction metadata, and multi-document attachment.
- **0004_gst_snapshot** (**0004_gst_snapshot.py**): Adds **gst_rate_percent** column to **quote_revisions** to snapshot the applicable tax rate per revision.

## 11. API & Interface Design

All endpoints require authentication via the **fq_session** HTTP-only cookie, except **POST /api/auth/login** and **GET /api/health**. All mutating requests (**POST**, **PATCH**, **DELETE**) require the **X-CSRF-Token** header.

### 11.1 Authentication & Session Endpoints

##### **POST /api/auth/login**

- Purpose: Authenticate user credentials, establish a 12-hour server session, and issue CSRF token.
- Access Control: Public.
- Request Schema:

```json
{
"username": "estimator@demo.local",
"password": "estimate-demo"
```

}

```

• Response Schema (**200 OK**):

```json

{

"user": {

"id": 1,

"username": "estimator@demo.local",

"display_name": "Aarav · Estimator",

"role": "estimator"

},

"csrf_token": "a1b2c3d4e5f6g7h8i9j0k1l2"

}

```

• Set-Cookie Header: **fq_session=<token>; HttpOnly; SameSite=Strict; Max-Age=43200; Path=/**

##### **POST /api/auth/logout**

• Purpose: Invalidate the active session on the server and expire the browser cookie.

• Access Control: Authenticated. Requires **X-CSRF-Token**.

• Response (**200 OK**): **{"ok": true}**

##### **GET /api/me**

• Purpose: Retrieve active user profile and refresh CSRF token on application boot.

• Access Control: Authenticated.

• Response (**200 OK**): User payload + **csrf_token**.

### 11.2 RFQ & Costing Endpoints

##### **GET /api/rfqs**

• Purpose: List all RFQs with owner, part count, latest revision value, and overdue calculation.

• Parameters: **q** (string, optional search filter), **status** (string, optional status filter).

• Response Schema (**200 OK**):

```json

{

"items": [

{

"id": 1,

"rfq_number": "RFQ-2611-014",

"customer": "Company B (two-wheelers)",

"received_date": "2026-11-07",

"due_date": "2026-11-11",

"owner": "Aarav · Estimator",

"owner_id": 1,

"status": "estimating",

"loss_reason": null,

"synthetic": false,

"source_label": "Provided sample · demo dates and rates",

"part_count": 1,

"sources": [],

"annual_value": 19140000.0,

"revision_id": 1

}

],

"total": 1,

"overdue": 0

}

```

##### **POST /api/rfqs**

- Purpose: Create a new multi-part RFQ package linked to uploaded source documents.
- Access Control: Estimator role only. Requires **X-CSRF-Token**.
- Request Schema:

```json

{

"rfq_number": "RFQ-2026-C45-099",

"customer": "Mahindra Powertrain",

"due_date": "2026-11-20",

"parts": [

{

"part_name": "Brake Cam Shaft",

"material": "C45",

"gross_weight_kg": 0.82,

"annual_volume": 60000,

"cycle_time_min": 3.8,

"machine_rate_key": "General machining",

"source_ref": "Drawing REV-B, Note 4"

}

],

"source_file_ids": ["034677c3e17448edbc6026bfebaeaa74.pdf"]

}

```

- Error Responses: **409 Conflict** (RFQ number exists), **422 Unprocessable Entity** (Missing parts or duplicate source files).

##### **PATCH /api/parts/{part_id}**
```

- Purpose: Update part costing parameters and multi-step machine routing operations.
- Access Control: Estimator role only. Requires **X-CSRF-Token**.
- Request Schema:

```json
{
"gross_weight_kg": 0.85,
"scrap_pct": 2.5,
"overhead_per_part": 1.5,
"freight_per_part": 0.8,
"packing_per_part": 0.5,
"margin_pct": 18.0,
"routing_operations": [
{
"operation_name": "Billet Shear & Forge",
"machine_rate_key": "Forging machining",
"cycle_time_min": 2.2
},
{
"operation_name": "Finish CNC Turning",
"machine_rate_key": "Turning cell",
"cycle_time_min": 1.6
}
]
}
```

##### **POST /api/rfqs/{rfq_id}/cost**

- Purpose: Execute deterministic quote calculation across 2 or 3 volume scenarios, snapshot current rates, and create a draft revision.
- Access Control: Estimator role only. Requires **X-CSRF-Token**.
- Request Schema:

```json
{
"volumes": [10000, 50000, 180000]
}
```

- Response Schema (**200 OK**):

```json
{
"revision_id": 2,
```

"revision_number": 1,
"status": "draft",
"annual_value": 19140000.0,
"reference_volume": 180000,
"breakdown": [
{
"part_id": 1,
"part_name": "Rear wheel spindle",
"material": "EN8",
"routing_operations": [
{
"operation_name": "Primary operation",
"machine_rate_key": "Turning cell",
"cycle_time_min": 3.2,
"rate_amount": 850.0,
"rate_unit": "₹ / hour"
}
],
"scenarios": [
{
"volume": 10000,
"material": 71.98,
"processing": 45.33,
"overhead": 0.0,
"freight": 0.0,
"packing": 0.0,
"unit_cost": 117.31,
"unit_price": 138.43,
"annual_value": 1384300.0
},
{
"volume": 180000,
"material": 71.98,
"processing": 45.33,
"overhead": 0.0,
"freight": 0.0,
"packing": 0.0,
"unit_cost": 117.31,
"unit_price": 138.43,

"annual_value": 24917400.0
}
],
"margin_pct": 18.0
}
],
"rate_snapshots": [
{
"part_id": 1,
"material_rate_id": 1,
"material_rate": 118.0,
"material_rate_key": "EN8",
"material_unit": "₹ / kg",
"machine_rate_id": 9,
"machine_rate": 850.0,
"machine_rate_key": "Turning cell",
"machine_unit": "₹ / hour"
}
],
"gst_rate_percent": 0.0,
"gst_annual_value": 0.0,
"annual_value_including_gst": 24917400.0
}
```

### 11.3 Sales Governance & Review Endpoints

##### **POST /api/revisions/{revision_id}/submit**

- Purpose: Transition revision from **draft** to **pending_approval**.
- Access Control: Estimator role only. Requires **X-CSRF-Token**.

##### **GET /api/approvals**

- Purpose: Retrieve all quote revisions currently awaiting Sales Head review.
- Access Control: Sales Head role only.

##### **POST /api/revisions/{revision_id}/decision**

- Purpose: Approve or return a quote revision. If returning, **reason** is mandatory.
- Access Control: Sales Head role only. Requires **X-CSRF-Token**.
- Request Schema (Return):

```json
{
"decision": "return",
"reason": "Customer target price is ₹125/pc; reduce cycle time on finish grinding."
```
```

}

```

• Request Schema (Approve):

```json

{

"decision": "approve"

}

```

• Response (**200 OK**):

```json

{

"status": "approved",

"threshold_flag": true

}

```

##### **GET /api/revisions/{revision_id}/pdf**

• Purpose: Stream the official quotation cost-breakup PDF document.

• Access Control: Authenticated. Allowed only if revision status is **approved** or **sent**.

• Response: Binary stream with header **Content-Type: application/pdf**.

### 11.4 Ingestion & Historical Import Endpoints

##### **POST /api/intake/file-extract**

• Purpose: Upload RFQ document, persist file, run OCR/parser, and return suggested fields.

• Payload: **multipart/form-data** with **file** and optional **demo_only=true**.

##### **POST /api/imports/preview**

• Purpose: Inspect Excel workbook, list sheets, return detected headers and column mapping suggestions. If **mapping** JSON is supplied, validates data rows and writes temporary batch JSON.

• Access Control: Estimator role only. Requires **X-CSRF-Token**.

##### **POST /api/imports/{batch_id}/commit**

• Purpose: Atomically insert validated historical quote rows into **historical_quotes** and delete temporary batch JSON.

• Access Control: Estimator role only. Requires **X-CSRF-Token**.

## 12. AI/ML Architecture

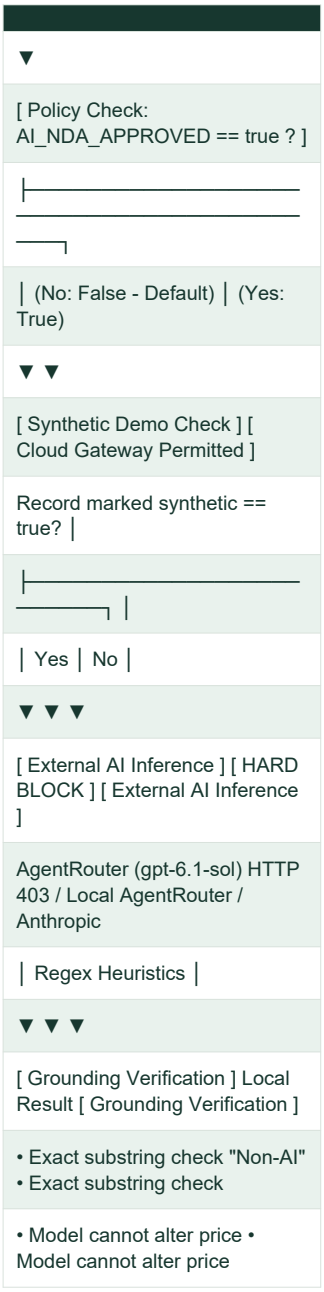
```

+----------------------------------------------------------------------------------------------+

| **AI/ML BOUNDARY ARCHITECTURE** |

+----------------------------------------------------------------------------------------------+

| |
| --- |
| [ User Action ] |
| \| |




+----------+

```

### 12.1 Implemented AI Capabilities

ForgeQuote integrates an assistive, non-autonomous AI pipeline designed strictly to eliminate manual data entry and accelerate historical research. The system implements three distinct AI capabilities:

1. Intake Document Information Extraction (**ai_extract**): Extracts listed RFQ parameters (**rfq_number**, **customer**, **due_date**, **part_name**, **material**, **gross_weight_kg**, **annual_volume**, **cycle_time_min**, **machine_rate**) from ingested document text.

2. Historical Routing Candidate Ranking (**routing_suggestions**): Ingests the top 30 attribute-matched historical parts and uses the model to rank the 5 most viable historical routings with recorded machine keys and cycle times.

3. Conversational Estimation Assistant (**assistant_chat**): A context-aware RAG assistant providing natural-language answers to questions like "What similar tractor parts have we quoted in 20MnCr5?" based strictly on retrieved historical quotes.

### 12.2 Model Selection & Gateway Rationale

- Supported Protocols: Configured dynamically via **AI_PROTOCOL** (**openai**, **responses**, or **anthropic**).
```

- Configured Target: Uses AgentRouter's OpenAI-compatible **/v1** endpoint targeting **gpt-6.1-sol** (or alternatively Anthropic's Messages endpoint with **claude-opus-4-8**).
- Rationale: AgentRouter provides a consolidated proxy interface for state-of-the-art reasoning models without requiring individual direct billing setups with multiple foundation model vendors during the prototype phase.

### 12.3 Data Privacy & Contractual NDA Boundary Enforcement

To comply with stringent OEM non-disclosure agreements:

- **AI_NDA_APPROVED** Setting: Defaults to **false** in **.env.example** and the runtime environment.
- Customer Data Isolation: Whenever **AI_NDA_APPROVED=false**, any attempt to trigger AI chat on an RFQ containing actual customer data is blocked with HTTP 403 Forbidden.
- File Upload Guard: During file intake (**POST /api/intake/file-extract**), external LLM processing is bypassed unless the user explicitly checks **demo_only=True** confirming that the document contains synthetic sample data. When unapproved, intake immediately executes **rule_extract()** (local regex heuristics) and labels suggestions as **"Local pattern suggestions only; no AI model ran"**.

### 12.4 Anti-Hallucination & Exact Substring Grounding

LLM hallucinations are mitigated through strict post-processing validation in **ai_extract()**:

```python

# The model must return an exact source excerpt alongside every proposed value

if name in allowed and value and excerpt and excerpt.casefold() in text.casefold():

accepted.append({

"name": name,

"value": value,

"source": {"file": filename, "excerpt": excerpt},

"confidence": float(field.get("confidence", 0.5)),

"status": "suggested",

"method": f"AI model: {AI_MODEL}"

})
```

Any field whose **source_excerpt** cannot be found verbatim within the source document is discarded.

### 12.5 What AI Is Explicitly Forbidden from Doing

- AI CANNOT Calculate Prices: Models are never asked or permitted to perform arithmetic costing, yield scrap subtraction, or margin applications.
- AI CANNOT Approve Quotes: All quote transitions require explicit authenticated human action.
- AI CANNOT Invent Operations: In routing suggestions, the model is strictly restricted to ranking supplied historical IDs; it cannot fabricate machine centers or cycle times.

## 13. Security Architecture

### 13.1 Authentication & Session Management

- Password Hashing: Passwords are never stored in plaintext. They are digested using PBKDF2-HMAC-SHA256 with 310,000 iterations and a 16-byte cryptographically secure random salt generated via **secrets.token_bytes(16)**.
- Session Tokens: Sessions are generated with 32 bytes of URL-safe entropy (**secrets.token_urlsafe(32)**).
- Cookie Security: Sessions are delivered in the **fq_session** cookie:

- **HttpOnly**: Inaccessible to browser JavaScript, preventing token exfiltration via XSS.
- **SameSite=Strict**: Cookies are withheld on cross-site requests, mitigating cross-site request forgery.
- **Path=/**: Uniformly available across the application.
- **Secure**: Toggled via the **COOKIE_SECURE** environment variable (must be **true** in production).
- **Max-Age=43200**: Sessions automatically expire after 12 hours.

### 13.2 Authorization & Role-Based Access Control (RBAC)

Role separation is strictly enforced at the API route handler level using **require_role(user, *roles)**:

| Role | Permitted Actions | Prohibited Actions |
|---|---|---|
| **estimator** | Create RFQ, edit owned RFQ headers, update part cost inputs and routing operations, calculate quote scenarios, submit quote revisions for review, upload RFQ source files, run intake extractions, preview and commit historical quote imports. | Approve quote revisions, return quote revisions, download approved quote PDFs, mark quotes sent, record commercial win/loss outcomes, reassign RFQ owners, create or edit rate versions. |
| **sales** | View approvals queue, approve quote revisions, return quote revisions (with mandatory reason), download approved quotation PDFs, mark quotes sent, record commercial win/loss outcomes, reassign RFQ owner to any estimator, edit any RFQ header, create and update rate master versions. | Calculate new quote revisions, modify part raw weights or machining cycle times. |

### 13.3 CSRF Defense Architecture

Every session generates a secondary, independent 24-byte random token (**session["csrf"]**). Mutating HTTP requests (**POST**, **PATCH**, **DELETE**) must supply this value in the **X-CSRF-Token** header. Validation is executed using Python's **hmac.compare_digest(session["csrf"], supplied)** to prevent timing side-channel attacks.

### 13.4 Threat Modeling & Attack Surface Analysis

| Threat / Attack Vector | Target Asset | Severity | Existing Mitigation | Recommended Future Hardening |
|---|---|---|---|---|
| Cross-Site Scripting (XSS) | Session Cookies | High | Session tokens isolated in **HttpOnly** cookies; all rendered strings sanitized via **esc()** utility. | Implement Content Security Policy (CSP) headers (**default-src 'self'**). |
| CSRF State Mutation | Quote Approval / Rate Creation | High | **SameSite=Strict** cookie policy + mandatory **X-CSRF-Token** header validated with constant-time comparison. | Enforce per-request nonce rotation for high-value actions (>₹25L). |
| Credential Stuffing | Sign-In API | Medium | PBKDF2-HMAC-SHA256 (310k iterations). | Implement IP-based and username-based rate limiting via Redis/SlowAPI. |
| Malicious Drawing Upload | Server Filesystem | High | File upload cap enforced at 12 MB; allowed file extensions strictly whitelisted; files renamed to UUID hex strings. | Execute file uploads inside an isolated sandboxed antivirus container (ClamAV). |
| Confidential Data Exfiltration | Customer Blueprints / RFQs | Critical | Hardcoded **AI_NDA_APPROVED** check; customer RFQ inference returns HTTP 403; local regex fallback. | Enterprise DLP (Data Loss Prevention) scanning and tokenization of customer names. |
| Denial of Service (DoS) | Tesseract OCR / PDF Engine | Medium | OCR limited to first 20 pages of scanned PDFs; maximum upload size 12 MB. | Offload OCR and PDF generation to asynchronous background Celery worker tasks. |

## 14. Error Handling & Resilience

### 14.1 Exception Handling Strategy

ForgeQuote uses structured HTTP exception handling. Specific error classes are mapped directly to semantic HTTP status codes:

- **HTTP 401 Unauthorized**: Missing, expired, or invalid session token. Handled by frontend SPA by clearing local state and displaying the sign-in modal.
- **HTTP 403 Forbidden**: Role permission violation (e.g., estimator attempting to approve a quote, or estimator attempting to edit an RFQ owned by another estimator) or NDA privacy violation.
- **HTTP 404 Not Found**: Entity not found in database or missing binary file.
- **HTTP 409 Conflict**: Business rule collision, such as attempting to create an RFQ number that already exists, creating an overlapping rate period, or approving a revision that is no longer in **pending_approval** status.
- **HTTP 413 Payload Too Large**: Uploaded file exceeds the 12 MB ceiling (**len(payload) > MAX_UPLOAD**).
- **HTTP 415 Unsupported Media Type**: Uploaded file extension is outside the approved whitelist (**.txt**, **.eml**, **.csv**, **.xlsx**, **.pdf**, **.png**, **.jpg**, **.jpeg**, **.tif**, **.tiff**, **.bmp**).
- **HTTP 422 Unprocessable Entity**: Semantic validation failure (e.g., negative cycle time, scrap $\ge 100\%$, missing return reason, or invalid column mapping).
- **HTTP 502 Bad Gateway**: Supabase Storage or external AI gateway failure.
- **HTTP 503 Service Unavailable**: AI gateway unconfigured, local Tesseract binary missing, or external gateway timeout.

### 14.2 Graceful Degradation & Fallbacks

- AI Gateway Unavailability: If the external AI service returns HTTP 4xx/5xx or times out, the intake pipeline gracefully falls back to local regex extraction (**rule_extract**), routing suggestions fall back to attribute matching, and the health endpoint reports **"ai": "unavailable"**.
- Corrupt Document Ingestion: If an uploaded Excel file or PDF is corrupt, **extract_intake_file()** captures the exception, logs **source.extraction_error**, retains the raw file in storage, and returns an empty suggestions list, allowing the estimator to enter parameters manually.
- Storage Redundancy: If Supabase Storage credentials are unset, the system logs an operational warning and seamlessly stores files under **instance/uploads/**.

## 15. Performance & Scalability

### 15.1 Computational Complexity & Query Optimization

- Costing Engine: The core calculation runs in $O(P \times R \times V)$ where $P$ is the number of parts (typically $1 \le P \le 10$), $R$ is routing operations ($1 \le R \le 12$), and $V$ is volume scenarios ($2 \le V \le 3$). Maximum iterations per calculation: $10 \times 12 \times 3 = 360$ operations, executing in under $15\text{ ms}$.
- Eager Relationship Loading: The application mitigates SQLAlchemy N+1 query overhead by explicitly applying **joinedload()** on critical foreign key relationships:

```python
query = db.query(RFQ).options(
joinedload(RFQ.owner),
joinedload(RFQ.parts),
joinedload(RFQ.revisions),
joinedload(RFQ.sources)
)
```

- Historical Search Optimization: History search queries execute case-insensitive matching across indexed columns (**part_name**, **material**, **rfq_number**, **customer**) and apply SQL **LIMIT 12** during RAG retrieval and **page_size** slicing during historical browsing.

### 15.2 Scalability Bottlenecks & Evolution Path

1. In-Memory Session Store:

- Current State: **SESSIONS: dict[str, dict]** resides in the single Python process memory.
- Bottleneck: Multi-worker deployments (**uvicorn --workers 4**) or multi-pod container clusters will cause random session loss.
- Target Solution: Migrate session storage to Redis via **redis-py** or persist sessions in a dedicated PostgreSQL **user_sessions** table.

2. Synchronous OCR & Workbook Import:

- Current State: Tesseract OCR and openpyxl imports execute within the HTTP request thread.
- Bottleneck: A 20-page high-DPI scanned PDF can block worker threads for 4 to 8 seconds.
- Target Solution: Introduce an asynchronous task queue (Celery or ARQ) backed by Redis, returning a task ID and polling/WebSocket notifications for completion.

3. Local Temporary Import Staging:

- Current State: Excel import previews write temporary batch JSON files to **instance/uploads/import-{token}.json**.
- Bottleneck: Multi-instance deployments behind a round-robin load balancer cannot guarantee that the **commit** request hits the instance holding the staging file.
- Target Solution: Store import staging records in a temporary database table (**import_batches** and **import_batch_rows**) or Redis key with automatic TTL.

## 16. Deployment Architecture

### 16.1 Deployment Topology

```mermaid
graph TB
subgraph IngressTier ["Ingress & Load Balancing"]
ALB["Cloud Load Balancer / Reverse Proxy (Nginx / AWS ALB)<br/>• HTTPS Termination (Port 443)<br/>• Strict Security Headers (HSTS, CSP)<br/>• Compression (gzip/brotli)"]
end
subgraph AppCluster ["Containerized Application Cluster (ECS / Kubernetes)"]
direction LR
Pod1["FastAPI Container 1<br/>(uvicorn --workers 2)"]
Pod2["FastAPI Container 2<br/>(uvicorn --workers 2)"]
PodN["FastAPI Container N<br/>(Autoscaling Group)"]
end
subgraph StateTier ["State & Queue Tier"]
RedisCluster[("Redis Enterprise / AWS ElastiCache<br/>• Distributed Sessions (fq_session)<br/>• Rate Limiting Counters<br/>• Celery Task Queue Broker")]
end
subgraph WorkerCluster ["Asynchronous Background Workers"]
Worker1["Celery OCR Worker<br/>(Tesseract + Ghostscript)"]
Worker2["Celery Batch Import Worker<br/>(openpyxl streaming)"]
end
subgraph DatabaseTier ["Managed Cloud Persistence (Supabase / AWS RDS)"]
PrimaryDB[("PostgreSQL 15+ Primary<br/>• ACID Transactions<br/>• PgBouncer Connection Pool")]
ReplicaDB[("PostgreSQL Read Replica<br/>• Historical Search & Reporting")]
end
subgraph StorageTier ["Cloud Object Storage"]
CloudBucket["Supabase Private Storage Bucket<br/>(rfq-source-documents)<br/>Server-Side Encryption"]
end
ALB --> Pod1
ALB --> Pod2
ALB --> PodN
Pod1 --> RedisCluster
Pod2 --> RedisCluster
PodN --> RedisCluster
Pod1 --> PrimaryDB
Pod2 --> PrimaryDB
PodN --> PrimaryDB
Pod1 --> CloudBucket
```

Pod2 --> CloudBucket

PodN --> CloudBucket

RedisCluster --> Worker1

RedisCluster --> Worker2

Worker1 --> CloudBucket

Worker2 --> PrimaryDB

```

### 16.2 Containerization Specification (Recommended Dockerfile)

```dockerfile

## Multi-stage production container build

FROM python:3.12-slim-bookworm AS base

ENV PYTHONUNBUFFERED=1 \

PYTHONDONTWRITEBYTECODE=1 \

PIP_NO_CACHE_DIR=1 \

PORT=8000

## Install system dependencies: Tesseract OCR, English data, MuPDF C dependencies

RUN apt-get update && apt-get install -y --no-install-recommends \

tesseract-ocr \

tesseract-ocr-eng \

libmupdf-dev \

curl \

&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

## Install Python dependencies

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

## Copy application source code

COPY . .

## Run as non-root unprivileged service account

RUN useradd -m -u 1000 forgequote && \

mkdir -p /app/instance/uploads && \

chown -R forgequote:forgequote /app

USER forgequote

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \

```
CMD curl -f http://localhost:8000/api/health || exit 1

CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

### 16.3 Production Environment Configuration Reference

| Variable Name | Required | Default / Example Value | Security Classification | Operational Description |
|---|---|---|---|---|
| **DATABASE_URL** | Yes | **postgresql+psycopg://user:pass@host:5432/db** | Confidential | Primary PostgreSQL relational connection string. Requires TLS (**sslmode=require**). |
| **APP_ENV** | Yes | **production** | Public | Environment identifier (**production**, **staging**, **development**). |
| **SESSION_SECRET** | Yes | **[random 64-character hex string]** | Secret | Cryptographic master key for session token derivation. |
| **COOKIE_SECURE** | Yes | **true** | Public | Enforces **Secure** flag on **fq_session** cookie; requires HTTPS. |
| **GST_RATE_PERCENT** | Yes | **18.0** (or client-approved rate) | Operational | Global fallback GST percentage snapshotted into quote revisions. |
| **SUPABASE_URL** | Optional | **https://xxxx.supabase.co** | Public | Endpoint URL for Supabase project instance. |
| **SUPABASE_SECRET_KEY** | Optional | **sbp_secret_...** | Secret | Server-only administrative key for private storage bucket access. |
| **SUPABASE_STORAGE_BUCKET** | Optional | **rfq-source-documents** | Operational | Name of private object storage bucket. |
| **AI_BASE_URL** | Optional | **https://agentrouter.org/v1** | Public | Target AI gateway endpoint URL. |
| **AI_PROTOCOL** | Optional | **openai** | Operational | LLM wire protocol (**openai**, **responses**, **anthropic**). |
| **AI_MODEL** | Optional | **gpt-6.1-sol** | Operational | Foundation model identifier for extraction and chat. |
| **AI_API_KEY** | Optional | **sk-...** | Secret | Provider API key for hosted model gateway. |
| **AI_NDA_APPROVED** | Yes | **false** | Critical Policy | Controls data boundary: when false, customer data is barred from hosted inference. |
| **TESSERACT_CMD** | Optional | **/usr/bin/tesseract** | Operational | Path to Tesseract OCR executable if not on system PATH. |
| **OCR_LANGUAGE** | Optional | **eng** | Operational | Language training pack code for OCR processing. |
| **SEED_DEMO_DATA** | Optional | **false** | Operational | When true, populates synthetic demo records on startup (PostgreSQL only). |

## 17. Testing Strategy

### 17.1 Existing Automated Test Suite Analysis

ForgeQuote includes an automated test suite executed via **pytest**:

- **tests/test_costing.py** (Unit Tests):
- **test_sample_process_component_and_complete_cost**: Verifies processing cost calculation (₹45.33), unit cost (₹106.33), and annual volume total (₹19,140,000.00) against the brief's sample data.
- **test_invalid_boundaries_fail**: Parameterized boundary test verifying that zero gross weight, 100% scrap, and 100% margin raise **ValueError**.
- **test_invalid_volume_fails**: Verifies that zero or negative volume triggers input validation errors.
- **test_multiple_routing_operations_sum_process_cost**: Asserts that multi-operation process costs sum correctly (₹69.08 across Turning and Grinding).
- **tests/test_workflow.py** (Integration & Workflow Tests):
- **test_seeded_end_to_end_approval_and_pdf**: Full lifecycle validation: login $\rightarrow$ RFQ costing $\rightarrow$ multi-route update $\rightarrow$ submission $\rightarrow$ Sales login $\rightarrow$ owner reassignment $\rightarrow$ approval $\rightarrow$ ReportLab PDF generation $\rightarrow$ mark sent.
- **test_import_column_mapping_and_synthetic_provenance**: Validates dynamic column mapping and synthetic provenance marking.
- **test_routing_operations_repeated_update**: Validates that repeated updates to routing operations do not violate unique sequence constraints.
- **test_corrupt_file_extract_handled_gracefully**: Verifies that uploading corrupt binary payloads returns **method: "unavailable"** and retains file records without 500 errors.
- **test_history_paginates_complete_synthetic_dataset**: Validates pagination, total counts, and non-overlapping pages across 50+ records.
- **test_rfq_header_update_and_role_access**: Validates owner authorization boundaries on RFQ headers.
- **test_import_preview_lists_worksheets_and_can_select_one**: Validates multi-sheet workbook detection and targeted sheet selection.
- **test_gst_is_separate_and_snapshotted**: Validates that GST percentage is separately calculated and frozen in revision records.
- **test_1400_row_synthetic_workbook_import**: End-to-end stress test validating preview, validation, and batch commitment of 1,400 Excel rows in under 4 seconds.

### 17.2 Recommended Future Quality Gates

- Contract & Mutation Testing: Introduce **mutmut** to verify test suite robustness against subtle formula mutations.
- Load & Concurrency Testing: Implement Locust load testing scripts simulating 50 concurrent estimators calculating quotes simultaneously against a PostgreSQL container.
- Visual Regression Testing: Headless Playwright tests validating PDF pixel rendering and responsive layout across desktop (1920x1080) and tablet (768x1024) viewports.

## 18. Observability & Monitoring

### 18.1 Audit Trail Implementation

ForgeQuote maintains a synchronous, append-only relational audit log via the **audit_events** table:

```python
def audit(db: Session, user_id: int | None, action: str, entity: str, entity_id, detail: str = ""):
db.add(AuditEvent(actor_id=user_id, action=action, entity_type=entity,
entity_id=str(entity_id), detail=detail[:600]))
```

Audit records capture user identity, action type (**sign_in**, **create_rfq**, **calculate_quote**, **submit_for_approval**, **quote_approve**, **quote_return**, **download_quote_pdf**, **mark_quote_sent**, **rfq_won**, **rfq_lost**), target entity ID, and human-readable context.

### 18.2 System Health Endpoint (GET /api/health)

Provides live operational status:

```json
{
"status": "ok",
"database": "postgresql",
"ai": "reachable",
"file_storage": "supabase"
}
```

The endpoint dynamically detects database engine dialect, verifies Supabase Storage configuration, and reflects live AI gateway responsiveness (**reachable**, **unavailable**, **unverified**, or **not_configured**).

### 18.3 Recommended Production Monitoring & Alerting

- Metrics Collection: Integrate **prometheus-fastapi-instrumentator** to export standard RED metrics (Rate, Errors, Duration) to Prometheus and Grafana.
- Distributed Tracing: Implement OpenTelemetry SDK instrumentation to trace database transactions, Supabase Storage uploads, and AI gateway calls with trace propagation.
- Alerting Rules:
- P0 Critical: Database connection pool exhaustion (**pool_pre_ping** failure $> 1\%$ over 2 minutes).
- P1 High: Spindled AI gateway errors ($> 10\%$ failure rate over 5 minutes) triggering operational status degradation.
- P2 Warning: High rejection rate on historical workbook imports ($> 15\%$ row parsing errors).

## 19. Architecture Decision Records (ADR) Summary

Detailed ADR documents are maintained in [**docs/adr/**](file:///docs/adr/):

| ADR ID | Title | Status | Core Decision & Rationale | Source Path |
|---|---|---|---|---|
| ADR-001 | Deterministic Decimal Costing Engine | Accepted (Inferred) | Use Python **decimal.Decimal** with explicit **ROUND_HALF_UP** quantization to eliminate IEEE-754 floating-point drift in statutory quote calculations. AI models are strictly barred from pricing. | [**docs/adr/ADR-001-deterministic-decimal-costing-engine.md**](file:///docs/adr/ADR-001-deterministic-decimal-costing-engine.md) |
| ADR-002 | Dual Database Abstraction (SQLite & PostgreSQL) | Accepted (Inferred) | Abstract persistence via SQLAlchemy 2.0 and Alembic to support zero-dependency local SQLite development alongside cloud PostgreSQL (Supabase) with automated TLS enforcement. | [**docs/adr/ADR-002-dual-database-abstraction-sqlite-postgresql.md**](file:///docs/adr/ADR-002-dual-database-abstraction-sqlite-postgresql.md) |

| ADR ID | Title | Status | Core Decision & Rationale | Source Path |
|---|---|---|---|---|
| ADR-003 | Hybrid Storage (Local Disk vs. Supabase Storage) | Accepted (Inferred) | Store raw uploaded drawings and spreadsheets in private Supabase Storage buckets in production, falling back to local **instance/uploads/** during local review. | [**docs/adr/ADR-003-hybrid-storage-local-vs-supabase.md**](file:///docs/adr/ADR-003-hybrid-storage-local-vs-supabase.md) |
| ADR-004 | Data Privacy Boundaries & NDA Guardrails | Accepted (Inferred) | Enforce hard policy boundary via **AI_NDA_APPROVED**. When false, customer data is barred from hosted inference, defaulting to local regex extraction and local Tesseract OCR. | [**docs/adr/ADR-004-data-privacy-nda-guardrails-ai-gateway.md**](file:///docs/adr/ADR-004-data-privacy-nda-guardrails-ai-gateway.md) |
| ADR-005 | In-Memory Sessions & Double-Submit CSRF Defense | Accepted (Inferred) | Use 12-hour session tokens delivered in **HttpOnly**, **SameSite=Strict** cookies, paired with constant-time CSRF header verification. Flagged for Redis migration in production. | [**docs/adr/ADR-005-in-memory-session-and-csrf-protection.md**](file:///docs/adr/ADR-005-in-memory-session-and-csrf-protection.md) |
| ADR-006 | Multi-Operation Routing & Temporal Rate Masters | Accepted (Inferred) | Model industrial manufacturing via ordered **routing_operations** rows. Support non-overlapping effective date intervals for material and machine rate masters. | [**docs/adr/ADR-006-multi-operation-routing-and-rate-versioning.md**](file:///docs/adr/ADR-006-multi-operation-routing-and-rate-versioning.md) |
| ADR-007 | Statutory Governance & Quote Revision Immutability | Accepted (Inferred) | Enforce formal four-eyes quote lifecycle: draft $\rightarrow$ approval $\rightarrow$ dispatch $\rightarrow$ outcome. Snapshot all rates, breakdowns, and GST percentages to guarantee mathematical immutability. | [**docs/adr/ADR-007-statutory-sales-governance-and-quote-immutability.md**](file:///docs/adr/ADR-007-statutory-sales-governance-and-quote-immutability.md) |

## 20. Risks, Limitations & Mitigations

```

+-----------------------------------------------------------------------------------------------+

**RISK & MITIGATION MATRIX**

+-----------------------------------+------+--------+----------+---------------------------------+
```

| RISK DESCRIPTION | PROB | IMPACT | SEVERITY | MITIGATION STRATEGY |
|---|---|---|---|---|

```
+-----------------------------------+------+--------+----------+---------------------------------+
```

| 1. In-memory session loss on reboot | High | Medium | High | Migrate SESSIONS dict to Redis |
|---|---|---|---|---|
| 2. Sync OCR blocking HTTP threads | Med | High | High | Offload to Celery/ARQ workers |
| 3. Staged import lost across nodes | Med | Med | Medium | Store import staging in Redis/DB |
| 4. Noisy engineering drawing OCR | High | Med | Medium | Require human-in-the-loop review |
| 5. Upstream AI provider downtime | Med | Low | Low | Local regex extraction fallback |
| 6. Unapproved GST rate applied | Low | High | Medium | GST defaults to 0; manual sign-off |
| 7. Abandoned file upload sprawl | Med | Low | Low | S3/Supabase lifecycle cleanup rule |

+----------+------+--------+----------+----------+

```

| Risk Identifier | Category | Probability | Impact | Severity | Technical Description | Mitigation Strategy |
|---|---|---|---|---|---|---|
| RSK-01 | Architecture | High | Medium | High | In-memory session store (**SESSIONS = {}**) causes user sign-outs on server restart and prevents multi-worker autoscaling. | Migrate session management to Redis (**redis-py**) or database-backed session tables. |
| RSK-02 | Performance | Medium | High | High | Synchronous Tesseract OCR execution inside FastAPI endpoint can saturate worker threads during multi-page drawing uploads. | Offload OCR rendering to asynchronous background task queue (Celery/ARQ) with Redis broker. |
| RSK-03 | Scalability | Medium | Medium | Medium | Historical Excel import staging writes temporary JSON to local disk (**import-{token}.json**), failing on multi-container deployments. | Store import preview batches in Redis with 1-hour TTL or dedicated staging database tables. |
| RSK-04 | Operational | High | Medium | Medium | Low-resolution or hand-annotated engineering blueprints yield low OCR accuracy on title blocks and dimensions. | Require human-in-the-loop: extracted values are presented as unaccepted suggestions requiring estimator confirmation. |
```

| Risk Identifier | Category | Probability | Impact | Severity | Technical Description | Mitigation Strategy |
|---|---|---|---|---|---|---|
| RSK-05 | External Dependency | Medium | Low | Low | AgentRouter / OpenAI gateway latency spikes or rate limiting. | Automatic fallback to local pattern heuristics; strict 75-second timeout with circuit-breaking status flag. |
| RSK-06 | Compliance | Low | High | Medium | Quoting with unapproved GST percentage leads to client tax reconciliation disputes. | System enforces **GST_RATE_PERCENT=0** default and labels tax amounts explicitly on separate PDF invoice lines. |
| RSK-07 | Storage | Medium | Low | Low | Staged file uploads that are never committed to an RFQ create orphaned binary files. | Implement automated 24-hour cleanup cron job for unattached files in **rfq_sources** (**WHERE rfq_id IS NULL**). |

## 21. Future Architecture Roadmap

```

+----------------------------------------------------------------------------------------------+

**ENGINEERING ROADMAP TIMELINE**

+----------------------------------------------------------------------------------------------+

| |
|---|
| [ PHASE 1: SHORT-TERM (Weeks 1-4) ] |
| • Redis session store integration |
| • Docker & Docker Compose production manifests |
| • Asynchronous Celery background workers for OCR & Workbook Import |
| • Database migration to dedicated Supabase PostgreSQL instance |
| |
| [ PHASE 2: MEDIUM-TERM (Months 2-3) ] |
| • Managing Director / Executive sign-off chain for quotes > ₹50 Lakhs |
```

| |
|---|
| • Enterprise SSO integration (SAML 2.0 / Okta / Azure AD / OIDC) |
| • Client-specific tooling amortization and die-wear cost curve modeling |
| • Automated unattached upload garbage collection lifecycle job |
| |
| [ PHASE 3: LONG-TERM (Months 4-6) ] |
| • Bi-directional ERP connector (SAP S/4HANA / MS Dynamics Business Central) |
| • On-premises private LLM deployment (Ollama / vLLM with Llama-3-70B-Instruct) |
| • Direct 3D CAD parametric volume and surface area analyzer (STEP/IGES parser) |
| • Advanced Win/Loss predictive pricing recommendation engine |

+----------+

```

### 21.1 Short-Term Engineering Priorities (Weeks 1-4)

- Distributed Session Management: Replace the in-memory **SESSIONS** dictionary in **backend/security.py** with a Redis-backed session backend, enabling multi-pod Kubernetes horizontal autoscaling.
- Background Worker Extraction: Decouple **read_upload()** and **openpyxl** import parsing into asynchronous background worker tasks (Celery/Redis), keeping FastAPI request loops responsive.
- Production Containerization: Add production multi-stage Dockerfile and Docker Compose configurations including health checks, unprivileged service users, and volume mounts.
- Cloud Database Cutover: Apply migrations to production Supabase PostgreSQL and bootstrap named enterprise credentials via **python -m backend.bootstrap**.

### 21.2 Medium-Term System Enhancements (Months 2-3)

- Hierarchical Governance Matrix: Extend the approvals state machine to support multi-tiered executive sign-offs:
- Quotes $\le \text{₹25 Lakhs}$: Estimator $\rightarrow$ Sales Head.
- Quotes $>\text{₹25 Lakhs}$: Estimator $\rightarrow$ Sales Head $\rightarrow$ Plant Head.
- Quotes $>\text{₹1 Crore}$: Estimator $\rightarrow$ Sales Head $\rightarrow$ Plant Head $\rightarrow$ Managing Director.
- Enterprise SSO Integration: Implement OpenID Connect (OIDC) and SAML 2.0 adapters for corporate Azure AD / Okta identity providers.
- Tooling Amortization Engine: Allow estimators to model initial forging die tooling costs (e.g., ₹4,50,000) amortized across contractual part batch quantities.

### 21.3 Long-Term Strategic Architecture (Months 4-6)

- SAP / ERP Bi-directional Sync: Expose Webhook event buses and authenticated OData/REST endpoints to publish won quote BOMs directly into SAP S/4HANA production routing tables.
- Air-Gapped Private LLM Inference: Deploy a self-hosted, quantized vLLM instance (e.g., Qwen-2.5-72B or Llama-3.3-70B) inside Company A's private VPC, satisfying all customer NDA constraints without cloud reliance.
```

- Native CAD Geometry Ingestion: Integrate a server-side OpenCASCADE / PythonOCC parser to extract exact 3D bounding box volumes, projected die-parting areas, and estimated flash ratios directly from uploaded neutral STEP/IGES CAD files.

## 22. Appendix

### 22.1 Glossary of Manufacturing & Architectural Terms

- Closed-Die Forging: Manufacturing process where heated metal billets are shaped within machined impression dies under mechanical or hydraulic hammer presses.
- Billet: A semi-finished piece of raw alloy cylinder or bar stock cut to gross weight before forging.
- Flash Scrap: Excess raw material forced out into the die parting line gutter during forging impact, trimmed off in subsequent tooling.
- Yield Loss: The formula modeling gross raw material consumption adjusted for trimmed flash scrap: $C_{\text{mat}} = (W_{\text{gross}} \times R_{\text{mat}}) / (1 - \text{scrap})$.
- Machine Routing: The sequential order of machine centers and manufacturing operations required to convert a raw forged blank into a finished automotive part.
- Cycle Time: Total machine spindle and handling minutes required to complete an operation on a single part.
- Four-Eyes Principle: Financial and operational governance requirement mandating that quote revisions drafted by an Estimator must be independently reviewed and approved by the Sales Head.
- Statutory Threshold: The corporate governance rule mandating senior commercial executive review for quotes exceeding ₹25,00,000 (₹25 Lakhs) annual value.
- Deterministic Costing: Computational model guaranteeing identical arithmetic results regardless of computing environment, eliminating LLM hallucination and floating-point rounding divergence.
- ZDR (Zero Data Retention): Contractual SLA where an enterprise cloud provider commits to never store, log, or train on transmitted customer prompt data.

### 22.2 Repository File & Module Reference

| Absolute Repository File Path | Module Description | Architectural Significance |
|---|---|---|
| [**backend/main.py**](file:///backend/main.py) | Primary Application Server & REST API | Defines FastAPI routers, dependency guards, storage integrations, OCR pipelines, AI proxies, PDF generators, and historical importers. |
| [**backend/models.py**](file:///backend/models.py) | Relational ORM Data Models | Declares all 10 SQLAlchemy 2.0 mapped entity classes, constraints, foreign keys, and cascading relationships. |
| [**backend/costing.py**](file:///backend/costing.py) | Deterministic Cost Engine | Encapsulates the Python **Decimal** costing formulas, scrap adjustments, multi-routing cost summations, and half-up currency rounding. |
| [**backend/security.py**](file:///backend/security.py) | Cryptography & Session Manager | Implements PBKDF2 password hashing, session creation, in-memory state tracking, and constant-time CSRF verification. |
| [**backend/database.py**](file:///backend/database.py) | Database Connection & Engine Factory | Configures engine creation, dynamic dialect detection, connection pooling, and Supabase TLS enforcement. |
| [**backend/bootstrap.py**](file:///backend/bootstrap.py) | Production User Seeding CLI | Standalone administrative script for bootstrapping named production Estimator and Sales credentials securely without printing passwords. |
| [**backend/seed.py**](file:///backend/seed.py) | Demo Data Seeding Module | Populates 3 provided sample RFQs and 24 synthetic demo RFQs for local evaluation. |
| [**frontend/index.html**](file:///frontend/index.html) | Single-Page Application Shell | Semantic HTML5 structure for sign-in screens, topbars, sidebars, views, and modal dialog roots. |
| [**frontend/app.js**](file:///frontend/app.js) | SPA Frontend Client Logic | Manages reactive client state, view routing, asynchronous API orchestration, currency formatters, and intake modals. |
| [**frontend/styles.css**](file:///frontend/styles.css) | Core Design System Stylesheet | CSS3 custom properties, typography, responsive flex/grid layouts, neutral manufacturing palettes, and data tables. |
| [**migrations/versions/**](file:///migrations/versions/) | Versioned Alembic DDL Scripts | Linear schema evolution files (**0001_initial** through **0004_gst_snapshot**). |
| [**tests/test_costing.py**](file:///tests/test_costing.py) | Costing Engine Unit Test Suite | Regression assertions for formula precision, scrap yield calculations, and boundary value rejections. |
| [**tests/test_workflow.py**](file:///tests/test_workflow.py) | End-to-End Workflow Integration Tests | Automated test suite validating end-to-end quotation lifecycles, role permissions, PDF generation, and 1,400-row workbook imports. |

End of Technical Design Document (TDD.md) - ForgeQuote Enterprise Platform.